

Homework 9

DATA 624

Kevin Havis

November 22, 2025

Homework 8

8.1

To start we will set up some synthetic data to experiment with some tree models throughout this assignment.

```
set.seed(42)

simulated <- mlbench.friedman1(200, sd = 1)
simulated <- cbind(simulated$x, simulated$y)
simulated <- as.data.frame(simulated)
colnames(simulated)[ncol(simulated)] <- "y"
```

Part a

One of the useful parts about tree models is we can easily extract features importances to see which features the model weighted the most heavily. We will do this now with a simple Random Forest, an ensemble decision tree model.

```
model1 <- randomForest(y ~ ., data=simulated,
                       importance = TRUE,
                       ntree = 1000)

rfImp1 <- varImp(model1, scale = FALSE)

print(rfImp1)
```

	Overall
V1	8.61439539
V2	6.68772701
V3	2.02585114
V4	11.81770340
V5	2.53004882
V6	-0.08297300
V7	-0.08804903
V8	0.03412903
V9	0.24837034
V10	-0.18032337

The Random Forest model did not significantly use the uninformative predictors (V6-V10); they are given an importance < 1

Part b

Another advantage of tree models is that they are fairly robust to highly correlated features, although they will split the relative importance amongst them.

We will demonstrate this by creating a dummy value that is very highly correlated with one of the top features, and see how the model handles it

```
simulated$duplicate1 <- simulated$V1 + rnorm(200) * 0.1
cor(simulated$duplicate1, simulated$V1)
```

```
[1] 0.9554947
```

```
model2 <- randomForest(y ~ ., data=simulated,
                       importance = TRUE,
                       ntree = 1000)

rfImp2 <- varImp(model2, scale = FALSE)

print(rfImp2)
```

	Overall
V1	5.65711962
V2	6.22674125
V3	1.48494151

V4	10.70556478
V5	2.43026300
V6	-0.07969366
V7	-0.11173352
V8	0.01679566
V9	0.12389492
V10	-0.08672642
duplicate1	4.36641661

The importance value for V1 decreased from 8.61 without the outlier to 5.66 with the outlier. The model is finding them to be comparably important when splitting the dataset, so the importance is divided nearly equally.

```
# Add another outlier
simulated$duplicate2 <- simulated$V1 + rnorm(200) * 0.1

model3 <- randomForest(y ~ ., data=simulated,
                        importance = TRUE,
                        ntree = 1000)

rfImp3 <- varImp(model3, scale = FALSE)

print(rfImp3)
```

	Overall
V1	5.163586231
V2	6.455505756
V3	1.256726809
V4	11.071420826
V5	2.343093660
V6	-0.119153255
V7	-0.088897116
V8	-0.006225923
V9	0.177902376
V10	-0.068688662
duplicate1	3.236894802
duplicate2	2.373198259

V1 importance scores went down further, demonstrating the reduced information gain from V1 when we add highly correlated features; the importance is divided amongst the correlated features because they do an roughly equally good job at splitting the data.

Part c

A conditional forest model can be used to help reduce the impact of correlated features on importance scores. Let's see how this model handles the same data with correlated features.

```
model_cond <- party::cforest(y ~ ., data = simulated)

rfImp_cond <- varimp(model_cond, conditional = TRUE)

print(data.frame(rfImp_cond))
```

	rfImp_cond
V1	2.384363621
V2	4.128507363
V3	0.148231419
V4	6.461413387
V5	1.166745843
V6	-0.016916969
V7	-0.009031272
V8	0.028990701
V9	0.052854072
V10	0.012839425
duplicate1	1.316806119
duplicate2	0.743963976

For conditional model, the importances are roughly the same proportionally to the original random forest model, so I would concur that they do follow the same pattern.

Part d

Now we will compare several tree models in parallel to compare the feature importances selected by each variation.

```
# Set up training control
ctrl <- trainControl(
  method = "cv",
  number = 5,
  savePredictions = "final",
  classProbs = FALSE
)
```

```

# Train models
set.seed(42)

gbm_model <- train(
  y ~ .,
  data = simulated,
  method = "gbm",
  trControl = ctrl,
  tuneLength = 5,
  verbose = FALSE
)

rf_model <- train(
  y ~ .,
  data = simulated,
  method = "rf",
  trControl = ctrl,
  tuneLength = 5
)

cubist_model <- train(
  y ~ .,
  data = simulated,
  method = "cubist",
  trControl = ctrl,
  tuneLength = 5
)

# Extract feature importances
gbm_imp <- varImp(gbm_model)$importance %>%
  rownames_to_column("feature") %>%
  as_tibble() %>%
  mutate(model = "GBM") %>%
  rename(importance = Overall)

rf_imp <- varImp(rf_model)$importance %>%
  rownames_to_column("feature") %>%
  as_tibble() %>%
  mutate(model = "Random Forest") %>%
  rename(importance = Overall)

cubist_imp <- varImp(cubist_model)$importance %>%

```

```

rownames_to_column("feature") %>%
as_tibble() %>%
mutate(model = "Cubist") %>%
rename(importance = Overall)

# Combine and normalize by model
all_importances <- bind_rows(gbm_imp, rf_imp, cubist_imp) %>%
  group_by(model) %>%
  mutate(importance = importance / max(importance) * 100) %>%
  ungroup()

# Display importances
all_importances %>%
  arrange(model, desc(importance))

```

```

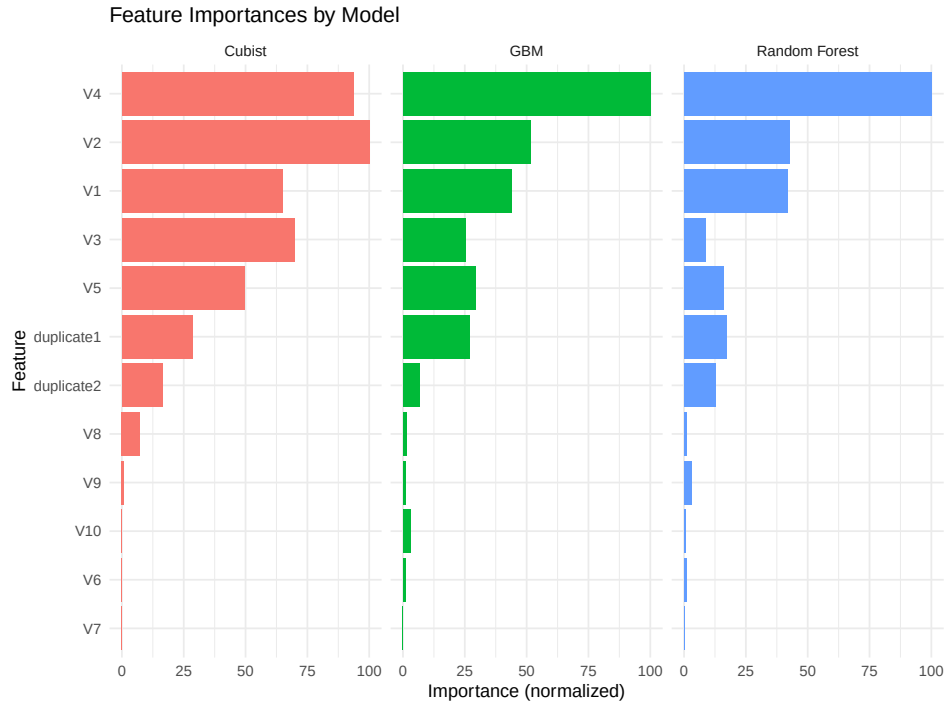
# A tibble: 36 x 3
  feature      importance model
  <chr>          <dbl> <chr>
1 V2             100    Cubist
2 V4             93.5    Cubist
3 V3             69.9    Cubist
4 V1             65.0    Cubist
5 V5             49.6    Cubist
6 duplicate1     28.5    Cubist
7 duplicate2     16.3    Cubist
8 V8              7.32   Cubist
9 V9              0.813   Cubist
10 V7              0      Cubist
# i 26 more rows

```

```

# Visualize
all_importances %>%
  ggplot(aes(x = reorder(feature, importance), y = importance, fill = model)) +
  geom_col(position = "dodge") +
  facet_wrap(~model, scales = "free_x") +
  coord_flip() +
  labs(title = "Feature Importances by Model",
       x = "Feature", y = "Importance (normalized)") +
  theme_minimal() +
  theme(legend.position = "none")

```



Other tree models, including Cubist and Gradient boosted, tend to follow the same pattern of feature importance. The one exception is that the Cubist model has weighted V4 less heavily, and generally has spread its feature importance more evenly.

8.2

Decision trees and related models have two primary hypertuning parameters; their depth and their minimum split size (i.e. number of branches). As with all models, more complexity can result in overfitting, while too little can miss some of the signal in the data.

The classic bias-variance trade off can be demonstrated below, showing how tree complexity can improve accuracy up to a point, but then leads to overfitting and reduced generalizability.

```
library(tidyverse)
library(rpart)

# Set seed for reproducibility
set.seed(123)

# 1. Generate true underlying relationship
n <- 200
```

```

x <- runif(n, 0, 10)
y_true <- sin(x) + 0.1 * x

# 2. Function to generate noisy data and fit trees at different depths
simulate_trees <- function(n_sims = 100, max_depth = 5) {
  depths <- 1:max_depth

  results <- expand_grid(
    sim = 1:n_sims,
    depth = depths
  ) %>%
  mutate(
    predictions = map2(sim, depth, function(s, d) {
      set.seed(s)
      x_train <- runif(n, 0, 10)
      y_train <- sin(x_train) + rnorm(n, 0, 0.5)

      tree <- rpart(y_train ~ x_train,
                    control = rpart.control(maxdepth = d, minsplit = 5))

      pred_data <- tibble(x_train = seq(0, 10, length.out = 100))
      predict(tree, pred_data)
    }),
    x_eval = list(seq(0, 10, length.out = 100))
  ) %>%
  unnest(c(predictions, x_eval)) %>%
  mutate(y_true_eval = sin(x_eval) + 0.1 * x_eval)

  return(results)
}

# 3. Calculate bias and variance
results <- simulate_trees(n_sims = 100, max_depth = 6)

metrics <- results %>%
  group_by(depth, x_eval) %>%
  summarise(
    pred_mean = mean(predictions),
    pred_var = var(predictions),
    y_true = first(y_true_eval),
    bias = (pred_mean - y_true)^2,
    .groups = "drop"
  )

```



```

) %>%
group_by(depth) %>%
summarise(
  avg_bias = mean(bias),
  avg_variance = mean(pred_var),
  avg_mse = avg_bias + avg_variance,
  .groups = "drop"
)

# 4. Visualize results
p1 <- metrics %>%
pivot_longer(starts_with("avg_"), names_prefix = "avg_") %>%
ggplot(aes(depth, value, color = name, shape = name)) +
geom_point(size = 3) +
geom_line() +
labs(
  title = "Bias-Variance Tradeoff: Tree Depth",
  x = "Tree Depth (Granularity)",
  y = "Error Component",
  color = "Metric",
  shape = "Metric"
) +
scale_color_manual(values = c("avg_bias" = "#E69F00",
                              "avg_variance" = "#56B4E9",
                              "avg_mse" = "#CC79A7")) +

theme_minimal() +
theme(legend.position = "bottom")

# 5. Visualize predictions at different depths
example_sim <- results %>%
  filter(sim <= 20) %>%
  mutate(depth = factor(depth, labels = paste("Depth =", 1:6)))

p2 <- example_sim %>%
  ggplot(aes(x_eval, predictions, group = sim)) +
  geom_line(alpha = 0.3, color = "steelblue") +
  geom_line(aes(x = x_eval, y = y_true_eval, group = NA),
            color = "red", size = 1, data = example_sim %>% distinct(depth, x_eval, y_true_eval))
facet_wrap(~depth, nrow = 2) +
labs(
  title = "Predictions from 20 Simulations (Red = True Function)",
  x = "x",

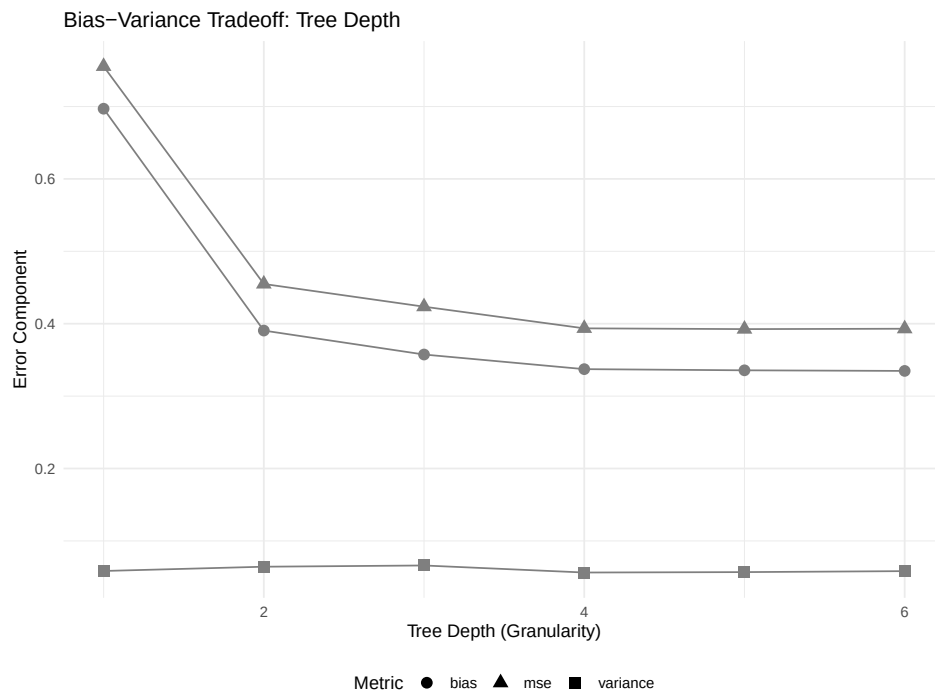
```

```

    y = "y"
) +
  theme_minimal()

print(p1)

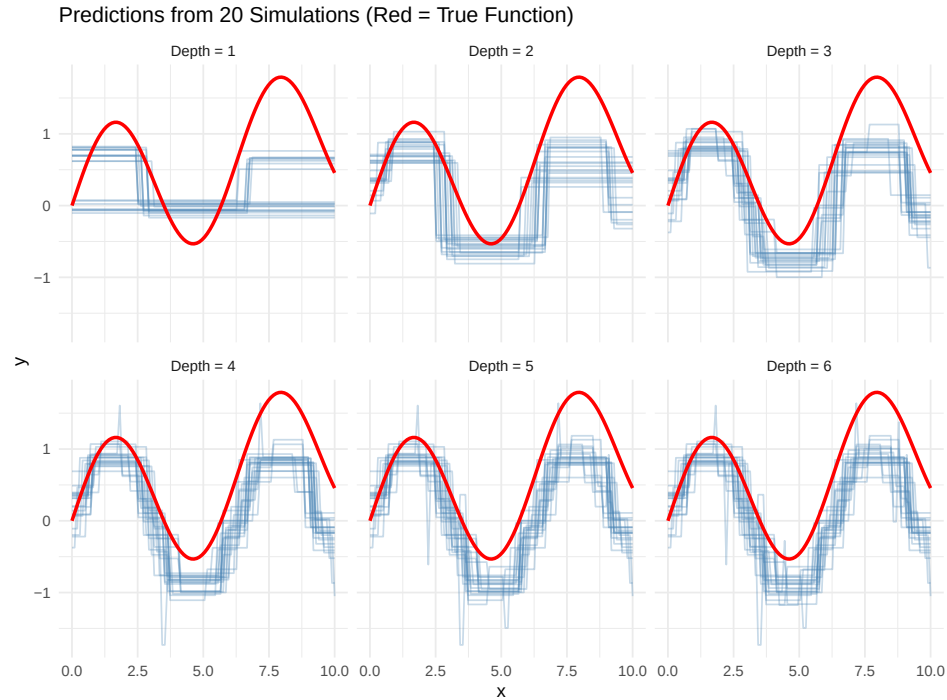
```



```

print(p2)

```



```
# 6. Print summary table
```

```
metrics %>%
  mutate(across(starts_with("avg_"), round, 4)) |> kable()
```

depth	avg_bias	avg_variance	avg_mse
1	0.6970	0.0585	0.7554
2	0.3905	0.0644	0.4549
3	0.3574	0.0661	0.4235
4	0.3372	0.0563	0.3936
5	0.3356	0.0569	0.3926
6	0.3348	0.0583	0.3930

8.3

Part a

The right hand model, with a higher learning rate and bagging fraction, is training more greedily due to these parameters. By setting the learning rate high, and the resample (bagging)

fraction to nearly 1, we've asked the right model to “trust” the early predictions of the boost stack more readily, and is also more exposed to a large portion of the training set. As such, it will produce trees that are biased towards the early “stumps”.

The left model is the inverse. We are allowing it to train very slowly, reducing the predictive value of each node, which results in a more aggregated prediction.

Part b

For additional test samples, I would expect the less greedy model (left) to be more performative. The greedy model has likely been overfit on the top feature (`NumCarbon`) and as such would be very reliant on that feature for predictive performance on out-of-sample data.

Part c

Increasing the interaction depth allows for features to interact with each other; as the depth of the tree increases, each node is dependant on all the nodes above it, hence creating and interaction effect between features. In this way, the relative importance of the features becomes more widely distributed, as the overall impact of a single feature is reduced by the later nodes using additional features to fit and predict.

Below is an example of how increasing the depth allows the importance to become more evenly distributed.

```
ctrl <- trainControl(  
  method = "cv",  
  number = 5,  
  savePredictions = "final",  
  classProbs = FALSE  
)  
  
# Train models  
set.seed(42)  
gbm_model_1 <- train(  
  y ~ .,  
  data = simulated,  
  method = "gbm",  
  trControl = ctrl,  
  tuneLength = 5,  
  verbose = FALSE  
)
```

```

gbm_model_2 <- train(
  y ~ .,
  data = simulated,
  method = "gbm",
  trControl = ctrl,
  tuneGrid = expand.grid(
    n.trees = 50:150,
    interaction.depth = 2,
    shrinkage = 0.1,
    n.minobsinnode = 10
  ),
  verbose = FALSE
)

gbm_model_3 <- train(
  y ~ .,
  data = simulated,
  method = "gbm",
  trControl = ctrl,
  tuneGrid = expand.grid(
    n.trees = 50:150,
    interaction.depth = 3,
    shrinkage = 0.1,
    n.minobsinnode = 10
  ),
  verbose = FALSE
)

gbm_model_4 <- train(
  y ~ .,
  data = simulated,
  method = "gbm",
  trControl = ctrl,
  tuneGrid = expand.grid(
    n.trees = 50:150,
    interaction.depth = 4,
    shrinkage = 0.1,
    n.minobsinnode = 10
  ),
  verbose = FALSE
)

```

```

# Extract feature importances
gbm_imp_1 <- varImp(gbm_model_1)$importance %>%
  rownames_to_column("feature") %>%
  as_tibble() %>%
  mutate(model = "GBM (depth = 1)") %>%
  rename(importance = Overall)

gbm_imp_2 <- varImp(gbm_model_2)$importance %>%
  rownames_to_column("feature") %>%
  as_tibble() %>%
  mutate(model = "GBM (depth = 2)") %>%
  rename(importance = Overall)

gbm_imp_3 <- varImp(gbm_model_3)$importance %>%
  rownames_to_column("feature") %>%
  as_tibble() %>%
  mutate(model = "GBM (depth = 3)") %>%
  rename(importance = Overall)

gbm_imp_4 <- varImp(gbm_model_4)$importance %>%
  rownames_to_column("feature") %>%
  as_tibble() %>%
  mutate(model = "GBM (depth = 4)") %>%
  rename(importance = Overall)

# Combine and normalize by model
all_importances <- bind_rows(gbm_imp_1, gbm_imp_2, gbm_imp_3, gbm_imp_4) %>%
  group_by(model) %>%
  mutate(importance = importance / max(importance) * 100) %>%
  ungroup()

# Display importances
all_importances %>%
  arrange(model, desc(importance))

```

```

# A tibble: 48 x 3
  feature      importance model
  <chr>         <dbl> <chr>
1 V4           100    GBM (depth = 1)
2 V2           51.5    GBM (depth = 1)
3 V1           44.0    GBM (depth = 1)
4 V5           29.5    GBM (depth = 1)

```

```

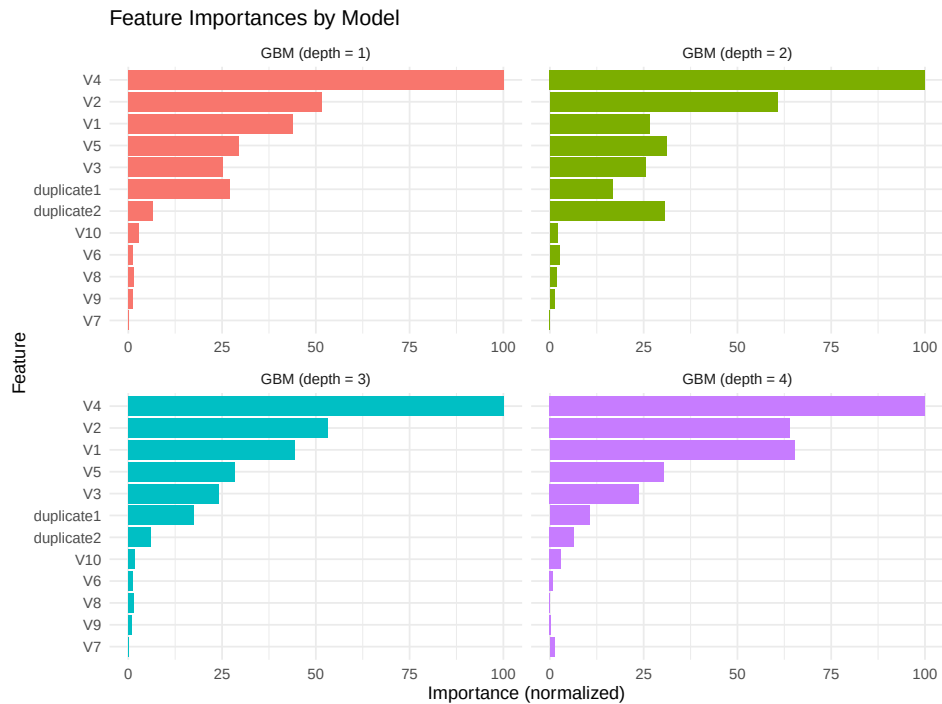
5 duplicate1      27.1 GBM (depth = 1)
6 V3              25.2 GBM (depth = 1)
7 duplicate2      6.50 GBM (depth = 1)
8 V10            2.92 GBM (depth = 1)
9 V8              1.48 GBM (depth = 1)
10 V9             1.21 GBM (depth = 1)
# i 38 more rows

```

```

# Visualize
all_importances %>%
  ggplot(aes(x = reorder(feature, importance), y = importance, fill = model)) +
  geom_col(position = "dodge") +
  facet_wrap(~model, scales = "free_x") +
  coord_flip() +
  labs(title = "Feature Importances by Model",
       x = "Feature", y = "Importance (normalized)") +
  theme_minimal() +
  theme(legend.position = "none")

```



8.7

Loading up the familiar dataset on manufacturing processes, we will now train tree models on the same data and review performance.

```
set.seed(42)

data(ChemicalManufacturingProcess)

# Split up the data set
y <- ChemicalManufacturingProcess[, 1]

x <- ChemicalManufacturingProcess[, -1]

# Use this partition to ensure well-distributed splits
idx <- createDataPartition(y, p = 0.8, list = FALSE)

train_x <- x[idx, ]
train_y <- y[idx]

test_x <- x[-idx, ]
test_y <- y[-idx]

# Use recipes to help us set up the preprocessing

rec <- recipe(y ~ ., data=data.frame(train_x, y=train_y)) |>
  step_impute_knn(all_predictors(), neighbors=5) #KNN null imputation
```

Part a

```
# Set up cross validation
chem_ctrl <- trainControl(method = 'cv', number = 5)

models <- caretList(
  x=train_x,
  y=train_y,
  preProcess = 'knnImpute',
  trControl = chem_ctrl,
```



```

methodList = c("gbm", "rf", "cubist"),
verbose = FALSE)

# Get predictions for all models
pred_gbm <- predict(models$gbm, newdata = test_x)
pred_rf <- predict(models$rf, newdata = test_x)
pred_cubist <- predict(models$cubist, newdata = test_x)

# Compare performance
results <- tibble(
  model = c("gbm", "rf", "cubist"),
  RMSE = c(postResample(pred_gbm, test_y)["RMSE"],
            postResample(pred_rf, test_y)["RMSE"],
            postResample(pred_cubist, test_y)["RMSE"]),
  Rsquared = c(postResample(pred_gbm, test_y)["Rsquared"],
               postResample(pred_rf, test_y)["Rsquared"],
               postResample(pred_cubist, test_y)["Rsquared"]),
  MAE = c(postResample(pred_gbm, test_y)["MAE"],
           postResample(pred_rf, test_y)["MAE"],
           postResample(pred_cubist, test_y)["MAE"])
)

results |> kable()

```

model	RMSE	Rsquared	MAE
gbm	1.1916983	0.6657837	0.9162817
rf	1.1743934	0.7372503	0.8574445
cubist	0.9917727	0.7788351	0.7400606

The Cubist model performs best across all metrics by a significant margin, with $R^2 = 0.7788$ and RMSE of 0.99.

Part b

```

cubist_importance <- varImp(models$cubist) |>
  pluck("importance") |>
  rownames_to_column("feature") |>
  as_tibble() |>

```

```

filter(Overall > 0) |>
  arrange(desc(Overall))

head(cubist_importance, 10) |> kable()

```

feature	Overall
ManufacturingProcess32	100.00000
ManufacturingProcess09	52.38095
ManufacturingProcess33	42.85714
ManufacturingProcess04	38.09524
BiologicalMaterial02	33.33333
ManufacturingProcess17	31.42857
BiologicalMaterial06	27.61905
BiologicalMaterial03	25.71429
ManufacturingProcess13	17.14286
ManufacturingProcess29	16.19048

Manufacturing processes were generally the most important for the Cubist model, specifically 32, 09, 33, and 04, followed by a mix of biological and manufacturing features.

These features importances are more similar to the SVM model we trained last time; both models listed 32 and 09 as the two most important. The same biological features are found to be informative for the two models as well.

The linear model did not share many important features with the Cubist model. This makes sense given Cubist is essentially non-linear way of finding patterns.

Part c

I was unable to find a way to plot the actual tree diagram for a Cubist model, but I've produced the model summary.

Reviewing the different linear equations the model selected, we can see that when used, **ManufacturingProcess32** always has a strong, positive coefficient towards the target variable.

From a business standpoint, this means that **ManufacturingProcess32** is likely a key driver of yield in the chemical manufacturing process, and optimizing this process could lead to significant improvements in overall yield.

```
summary(models$cubist$finalModel)
```

Call:

```
cubist.default(x = x, y = y, committees = param$committees, verbose = FALSE)
```

Cubist [Release 2.07 GPL Edition] Sat Nov 22 08:42:05 2025

Target attribute `outcome`

Read 144 cases (58 attributes) from undefined.data

Model 1:

Rule 1/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 0.930]

```
outcome = 40.183 + 1.45 ManufacturingProcess32
          + 1.02 ManufacturingProcess29 - 0.96 ManufacturingProcess25
          + 0.56 ManufacturingProcess09 - 0.48 ManufacturingProcess33
          + 0.33 ManufacturingProcess01 - 0.29 ManufacturingProcess17
          + 0.28 ManufacturingProcess39 + 0.23 BiologicalMaterial03
          - 0.22 BiologicalMaterial04
```

Model 2:

Rule 2/1: [7 cases, mean 39.030, range 37.3 to 40.96, est err 1.800]

```
if
ManufacturingProcess32 <= -1.689548
then
outcome = 14.099 - 13.7 ManufacturingProcess32
```

Rule 2/2: [137 cases, mean 40.221, range 35.25 to 44.35, est err 0.850]

```
if
ManufacturingProcess32 > -1.689548
then
outcome = 40.289 + 3.87 ManufacturingProcess18
          - 3.87 ManufacturingProcess20 + 1.43 ManufacturingProcess32
          + 0.71 ManufacturingProcess11 + 0.57 ManufacturingProcess19
```

+ 0.55 ManufacturingProcess04 - 0.52 ManufacturingProcess33
 - 0.47 ManufacturingProcess17 + 0.23 ManufacturingProcess09

Model 3:

Rule 3/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.043]

outcome = 40.197 + 1.37 ManufacturingProcess32
 + 1.15 BiologicalMaterial06 - 1.01 BiologicalMaterial02
 + 0.98 ManufacturingProcess09 + 0.64 ManufacturingProcess45
 - 0.55 ManufacturingProcess33 - 0.4 ManufacturingProcess35
 - 0.39 ManufacturingProcess37

Model 4:

Rule 4/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.039]

outcome = 40.076 - 1.07 ManufacturingProcess13
 + 0.76 ManufacturingProcess32 + 0.73 ManufacturingProcess15

Model 5:

Rule 5/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.108]

outcome = 40.503 + 2.09 ManufacturingProcess32
 - 1.12 ManufacturingProcess33 + 0.93 ManufacturingProcess09
 + 0.72 BiologicalMaterial03 - 0.55 BiologicalMaterial02
 + 0.54 ManufacturingProcess04

Model 6:

Rule 6/1: [108 cases, mean 39.750, range 35.25 to 44.35, est err 1.258]

if
 BiologicalMaterial06 <= 0.6861531
 then
 outcome = 39.607 + 2.44 BiologicalMaterial06 - 1.75 BiologicalMaterial02
 - 0.67 ManufacturingProcess13

Rule 6/2: [36 cases, mean 41.401, range 38.35 to 43.88, est err 1.290]

if
 BiologicalMaterial06 > 0.6861531

```

then
outcome = 44.503 - 2.26 BiologicalMaterial06
          + 1.38 ManufacturingProcess09 + 0.63 ManufacturingProcess39
          - 0.27 BiologicalMaterial10

```

Model 7:

Rule 7/1: [110 cases, mean 39.741, range 35.25 to 43.88, est err 1.250]

```

if
ManufacturingProcess17 > -0.7241278
then
outcome = 38.985 + 1.28 ManufacturingProcess32
          - 0.55 ManufacturingProcess33 - 0.47 BiologicalMaterial02
          + 0.44 BiologicalMaterial03 + 0.43 ManufacturingProcess09
          + 0.35 ManufacturingProcess04 - 0.23 ManufacturingProcess17

```

Rule 7/2: [76 cases, mean 40.893, range 35.25 to 43.88, est err 1.086]

```

if
BiologicalMaterial04 > -0.4627758
ManufacturingProcess32 > -0.3138902
then
outcome = 40.806 + 2.58 ManufacturingProcess32
          - 1.39 ManufacturingProcess33 + 0.94 ManufacturingProcess09
          + 0.77 ManufacturingProcess04 + 0.55 ManufacturingProcess45
          - 0.45 ManufacturingProcess28 - 0.17 BiologicalMaterial02
          + 0.16 BiologicalMaterial03

```

Rule 7/3: [34 cases, mean 41.529, range 37.89 to 44.35, est err 1.434]

```

if
ManufacturingProcess17 <= -0.7241278
then
outcome = 42.188 + 0.82 ManufacturingProcess32
          - 0.35 ManufacturingProcess33 - 0.3 BiologicalMaterial02
          + 0.28 BiologicalMaterial03 + 0.27 ManufacturingProcess09
          + 0.22 ManufacturingProcess04

```

Rule 7/4: [7 cases, mean 41.906, range 38.31 to 44.35, est err 4.496]

```

if
BiologicalMaterial04 <= -0.4627758

```

```

ManufacturingProcess32 > -0.3138902
then
outcome = 56.264 + 18.82 BiologicalMaterial04
          + 0.32 ManufacturingProcess32 - 0.19 ManufacturingProcess33
          + 0.12 ManufacturingProcess09 + 0.1 ManufacturingProcess04
          + 0.08 ManufacturingProcess45 - 0.07 ManufacturingProcess28

```

Model 8:

Rule 8/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.102]

```

outcome = 40.44 + 0.89 ManufacturingProcess32
          + 0.82 ManufacturingProcess15 + 0.63 ManufacturingProcess10
          - 0.53 ManufacturingProcess13

```

Model 9:

Rule 9/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.154]

```

outcome = 39.955 + 2.09 ManufacturingProcess32
          + 1.11 ManufacturingProcess09 - 1.09 ManufacturingProcess33
          + 0.63 ManufacturingProcess45 + 0.62 BiologicalMaterial03
          - 0.58 BiologicalMaterial06 + 0.57 ManufacturingProcess01

```

Model 10:

Rule 10/1: [64 cases, mean 39.338, range 35.25 to 43.88, est err 1.481]

```

if
BiologicalMaterial12 <= -0.3161927
then
outcome = 40.551 - 11.77 ManufacturingProcess39
          - 1.81 BiologicalMaterial12 + 0.86 BiologicalMaterial03
          + 0.62 ManufacturingProcess29 - 0.57 ManufacturingProcess26
          + 0.08 ManufacturingProcess32 - 0.07 BiologicalMaterial02
          - 0.06 ManufacturingProcess28 - 0.04 ManufacturingProcess13
          - 0.04 ManufacturingProcess17

```

Rule 10/2: [18 cases, mean 40.307, range 38.37 to 42.31, est err 1.473]

```

if
BiologicalMaterial12 > -0.3161927
ManufacturingProcess17 <= 0.1382921

```

```

ManufacturingProcess32 <= -0.1173676
then
outcome = 39.486 - 3.77 ManufacturingProcess17
          - 1.05 BiologicalMaterial12 + 0.38 ManufacturingProcess29
          - 0.35 ManufacturingProcess26 + 0.07 BiologicalMaterial03
          + 0.07 ManufacturingProcess32 - 0.06 BiologicalMaterial02
          - 0.06 ManufacturingProcess28 + 0.05 BiologicalMaterial05
          - 0.04 ManufacturingProcess13

```

Rule 10/3: [80 cases, mean 40.823, range 38.2 to 44.35, est err 1.134]

```

if
BiologicalMaterial12 > -0.3161927
then
outcome = 40.786 + 1.09 ManufacturingProcess32
          + 0.41 BiologicalMaterial08 - 0.39 BiologicalMaterial12
          - 0.39 ManufacturingProcess17 + 0.29 BiologicalMaterial05
          + 0.27 ManufacturingProcess29 - 0.25 ManufacturingProcess26

```

Rule 10/4: [38 cases, mean 41.123, range 35.25 to 44.16, est err 1.563]

```

if
ManufacturingProcess16 <= 0.1503436
ManufacturingProcess28 <= 0.7469431
ManufacturingProcess32 > -0.1173676
then
outcome = 40.858 + 13.95 ManufacturingProcess20
          - 6.01 ManufacturingProcess29 - 2.8 BiologicalMaterial02
          + 1.61 BiologicalMaterial03 - 1.44 ManufacturingProcess13
          - 0.97 ManufacturingProcess24 - 0.6 ManufacturingProcess28

```

Rule 10/5: [16 cases, mean 41.210, range 38.95 to 44.35, est err 2.219]

```

if
ManufacturingProcess16 > 0.1503436
ManufacturingProcess32 > -0.1173676
then
outcome = 37.957 + 4.16 ManufacturingProcess29
          - 3.44 ManufacturingProcess26 - 1.24 BiologicalMaterial02
          + 1.11 BiologicalMaterial03 + 0.82 ManufacturingProcess32
          - 0.79 ManufacturingProcess28 - 0.48 ManufacturingProcess13
          - 0.24 ManufacturingProcess24 + 0.15 BiologicalMaterial12
          - 0.11 BiologicalMaterial11

```

Rule 10/6: [26 cases, mean 41.851, range 36.83 to 44.35, est err 1.656]

```
if
ManufacturingProcess28 > 0.7469431
ManufacturingProcess32 > -0.1173676
then
outcome = 41.218 + 1.78 ManufacturingProcess29
          + 1.12 ManufacturingProcess32 + 1.04 BiologicalMaterial12
          - 0.73 BiologicalMaterial02 - 0.64 ManufacturingProcess28
          + 0.59 BiologicalMaterial03 - 0.45 BiologicalMaterial11
          - 0.39 ManufacturingProcess24 - 0.29 ManufacturingProcess13
          - 0.27 ManufacturingProcess17
```

Model 11:

Rule 11/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.082]

```
outcome = 39.96 + 2.08 ManufacturingProcess29
          - 1.93 ManufacturingProcess27 + 0.99 ManufacturingProcess32
          + 0.81 ManufacturingProcess45 + 0.81 ManufacturingProcess09
          + 0.77 ManufacturingProcess04 - 0.74 ManufacturingProcess17
          + 0.45 ManufacturingProcess34
```

Model 12:

Rule 12/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.073]

```
outcome = 40.386 + 8.27 ManufacturingProcess26
          - 8.27 ManufacturingProcess25 + 1.1 ManufacturingProcess32
```

Model 13:

Rule 13/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.171]

```
outcome = 40.008 + 1.22 ManufacturingProcess32
          + 0.93 ManufacturingProcess09 + 0.78 BiologicalMaterial12
          - 0.73 ManufacturingProcess17 + 0.53 ManufacturingProcess39
          + 0.48 ManufacturingProcess04 - 0.38 ManufacturingProcess28
          - 0.34 BiologicalMaterial11
```

Model 14:

Rule 14/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.175]

outcome = 40.371 + 1.12 ManufacturingProcess32

Model 15:

Rule 15/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.295]

outcome = 40.076 + 1.81 ManufacturingProcess32
+ 1.76 ManufacturingProcess09 + 0.95 BiologicalMaterial06
- 0.95 ManufacturingProcess33 - 0.75 BiologicalMaterial02
+ 0.68 ManufacturingProcess04

Model 16:

Rule 16/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.173]

outcome = 40.363 + 1.13 ManufacturingProcess32

Model 17:

Rule 17/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.294]

outcome = 40.084 + 1.8 ManufacturingProcess32
+ 1.76 ManufacturingProcess09 + 0.95 BiologicalMaterial06
- 0.95 ManufacturingProcess33 - 0.75 BiologicalMaterial02
+ 0.68 ManufacturingProcess04

Model 18:

Rule 18/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.172]

outcome = 40.355 + 1.14 ManufacturingProcess32

Model 19:

Rule 19/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.247]

outcome = 40.008 + 1.83 ManufacturingProcess32
+ 1.79 ManufacturingProcess09 - 0.95 ManufacturingProcess33
+ 0.58 ManufacturingProcess04

Model 20:

Rule 20/1: [144 cases, mean 40.163, range 35.25 to 44.35, est err 1.172]

outcome = 40.355 + 1.14 ManufacturingProcess32

Evaluation on training data (144 cases):

Average error	0.658
Relative error	0.45
Correlation coefficient	0.85

Attribute usage:

Conds Model

11%	94%	ManufacturingProcess32
5%	28%	ManufacturingProcess17
5%	11%	BiologicalMaterial12
5%	24%	BiologicalMaterial06
3%	5%	BiologicalMaterial04
2%	13%	ManufacturingProcess28
2%		ManufacturingProcess16
	55%	ManufacturingProcess09
	45%	ManufacturingProcess33
	40%	ManufacturingProcess04
	35%	BiologicalMaterial02
	27%	BiologicalMaterial03
	18%	ManufacturingProcess13
	17%	ManufacturingProcess29
	17%	ManufacturingProcess45
	13%	ManufacturingProcess39
	11%	ManufacturingProcess26
	9%	ManufacturingProcess01
	9%	ManufacturingProcess15
	9%	ManufacturingProcess25
	6%	BiologicalMaterial11
	6%	ManufacturingProcess20
	5%	ManufacturingProcess10
	5%	ManufacturingProcess27
	5%	ManufacturingProcess34
	5%	ManufacturingProcess35
	5%	ManufacturingProcess37

4%	ManufacturingProcess11
4%	ManufacturingProcess18
4%	ManufacturingProcess19
3%	BiologicalMaterial05
3%	BiologicalMaterial08
3%	ManufacturingProcess24
1%	BiologicalMaterial10

Time: 0.1 secs